Semántica operacional

Unidades de programas

Modelos de ejecución

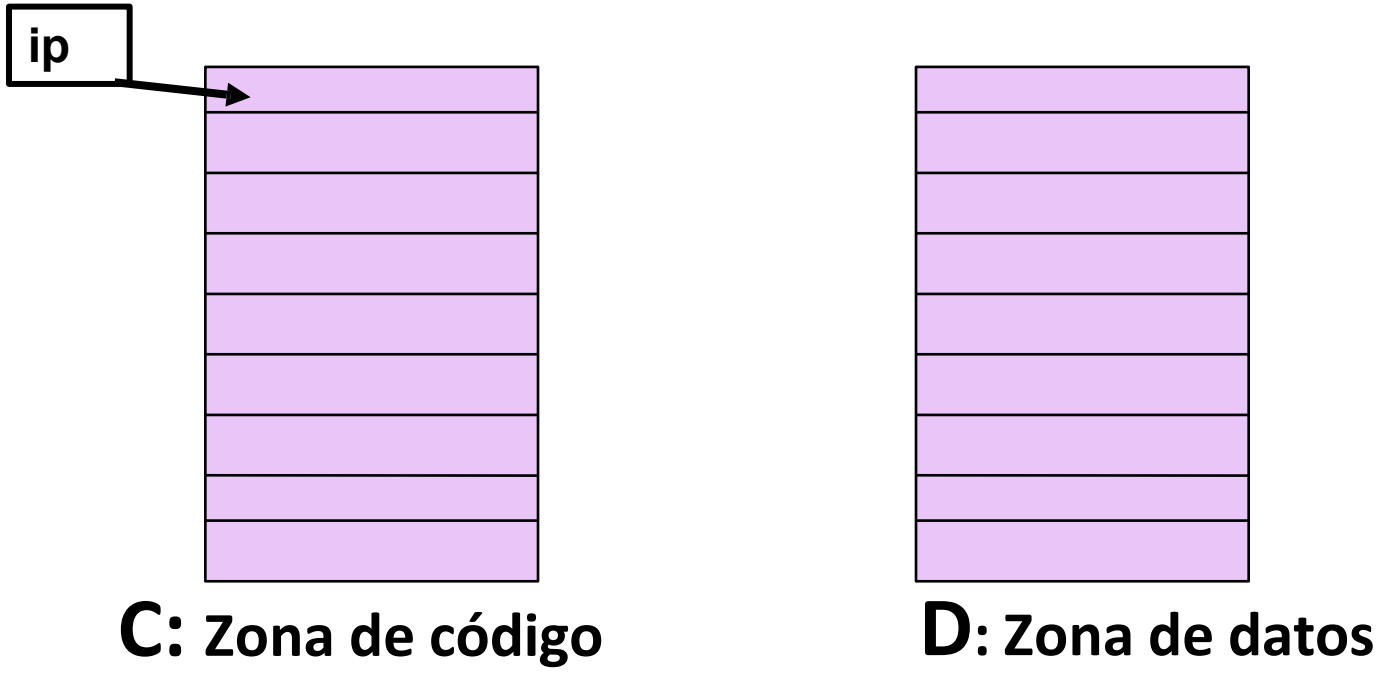
Unidades de programa
Repaso Clase Anterior

- **Atributos:**
  - **Nombre**
    - Declaración/invocación
  - **Alcance**
    - Dónde se conoce el identificador nombre Rutina
    - Ambiente de referencia de la Rutina: global, local, no local
  - **Tipo**
    - Dado por los parámetros y valor de retorno. **Signatura**
  - **L-Valor:**
    - Relacionado a dónde se almacena las sentencias en la memoria
  - **R-Valor:**
    - Relacionado al momento en que se hace la referencia a esa Rutina
- Diferencias entre **Declaración y Definición**
  - Extensión del alcance
- **Comunicación** entre Rutinas
  - Pasaje parámetros y entorno global

Procesador Abstracto - Memoria

Unidades

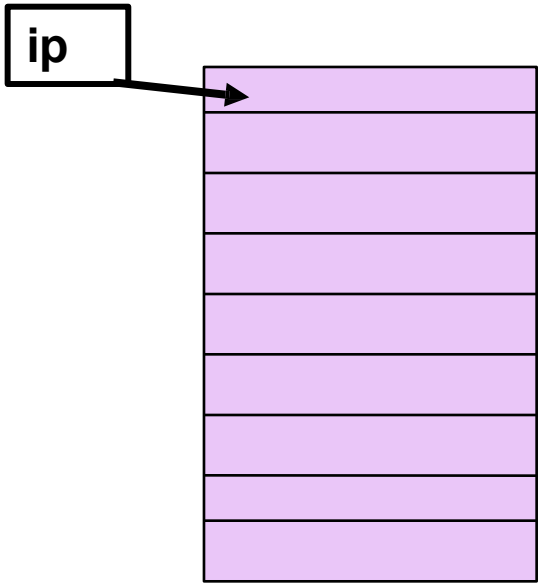
- Cuando se invoca una rutina se ejecuta una instancia del proceso con sus datos e información particular.
- La **instancia de la unidad** es la representación de la rutina en ejecución. Se tiene un registro de activación de la misma.
- El **ip** es el puntero a la instrucción siguiente a ejecutar en el código.



Procesador Abstracto - Memoria

Unidades

- En la zona de código están las sentencias.
- Se puede utilizar un lenguaje de un procesador abstracto como Simple SEM



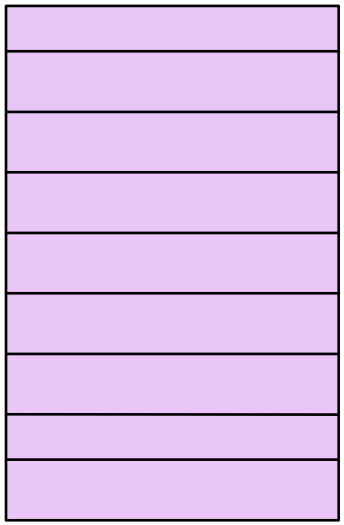
C: Zona de código

- **Set** Para asignar valores.
  Ej:
  set 10,D[20];
  set 20, read;
  set write,D[50]
- **Jump y jumpt** Bifurcación incondicional y condicional.
  Ej:
  jump 47,
  jumpt 47,D[13]>D[8]

Procesador Abstracto - Memoria

Unidades

- En la zona de datos se encuentra toda la información que maneja la unidad y que debe registrarse.
- El **registro de activación** es una estructura de datos en la memoria que se crea cada vez que se llama a una función o procedimiento en un programa, y contiene información esencial para la ejecución de dicha unidad, como sus parámetros de entrada, variables locales, y el punto de retorno al código que la llamó, cadena estática y dinámica, valor de retorno.



D: Zona de datos

Esquemas de ejecución de las unidades

- Estático
- Basado en pila
- Dinámico

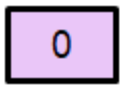
Repaso Clase Anterior Esquemas de ejecución -C1 y C2

- Esquemas de ejecución
  - Estático, basado en pila y dinámico
- **Estático**:
  - **C1**:
    - Programa sencillo sin bloques internos, solo código programa principal
    - En zona de datos: SOLO **datos locales**
  - **C2**:
    - Programa con bloques internos SIN anidamientos
    - Compilación junta y separada (**C2')**
    - En zona de datos: **datos locales + pto. de retorno**

C1

¿Cómo sería el **CÓDIGO?**

```
main()
 {
        int i, j;
        get(i, j);
        while (i != j)
                if (i > j)
                        i -= j;
                else
                        j -= i;
        print(i);
 }
```



C

0	set 0,read
1	net 1,read
2	jumpt 8, D[0] = D[1]
3	jumpt 6, D[0]<= D[1]
4	set 0, D[0] - D[1]
5	jump 7
6	set 1,D[1] – D[0]
7	jump 2
8	set write, D[0]
9	halt

D

0	celda reservada para i
1	celda reservada para j

C2: CALL-RETURN

¿Cómo cambia la información en la zona de **Datos**?

```
int i = 1, j = 2, k = 3;
alpha() {
   int i = 4, l = 5;
   ...
   i+=k+l;};
beta() {
   int k = 6;
   ...
   i=j+k;   alpha();
main() {
   ...
   beta();  }
```

1

Antes de saltar a Beta() guarda el punto de retorno

14	set 6,16	
15	jump 100	
49		
50		
58	set 4, D[4]+D[2]+D[5]	
99	jumpt D[3]	
100		
122	set 0,D[1] + D[7]	
123	set 3,125	
124	jump 50	
	jump D[6]	

seg.c main

seg.c alpha

seg.c beta

0	1	i	r.act. global
1	2	j	
2	3	k	
3		retorno	r.act. alpha
4	4	i	
5	5	l	
6	16	retorno	r.act. beta
7	6	k	

Guarda el punto de retorno al main

Esquemas de ejecución ESTÁTICO- C1 y C2

- Todo está cargado en la memoria
- 1 RA por unidad.
- Cada RA tiene:

Información que contiene como mínimo el RA:

- Datos locales
- Punto de retorno al código para cuando termine la unidad.
- Los datos no locales están todos en memoria por lo que se resuelven directamente todas las referencias a esos datos.

TIPO de variables según su TIEMPO DE VIDA: Solo ESTÁTICAS

Esquemas de ejecución
Casos de C3 a C6

- Estático
- Basado en pila
- Dinámico

C3
C2 + RECURSIÓN Y VALOR DE RETORNO

- Esquema basado en pila
- Rutinas con **capacidad de llamarse a sí mismas** (recursión directa) o de **llamar a otra rutina en forma recursiva** (recursión indirecta).
- Rutinas con la capacidad de **devolver valores**, es decir, funciones.

C3: Funcionamiento

- El registro de activación de cada unidad será de **tamaño fijo y conocido**, pero **no se sabrá cuantas instancias de cada unidad se necesitarán** durante la ejecución.
- Igual que en C2 el compilador puede ligar **cada variable con su desplazamiento dentro del correspondiente registro de activación**. El **desplazamiento** es **estático**
- **Cada nueva invocación aloca un nuevo registro de activación y se establecen las nuevas ligaduras entre el segmento de código y el nuevo registro de activación.**
- La **dirección** donde se cargará el **registro de activación**, es **dinámica**, por lo tanto, la **ligadura con la direcciones absolutas** en la zona de Datos de la memoria, **solo puede hacerse en ejecución**.

C3: FUNCIONAMIENTO

- La **dirección de las variables <u>se resuelven cuando se aloca el RA</u> donde se aplican los desplazamientos conocidos estáticamente**
- La **<u>ligadura</u> con la <u>direcciones</u> <u>absolutas</u> de cada variable** en la zona de Datos de la memoria, **<u>solo</u> puede hacerse <u>en ejecución</u>**.

A

var x,y: integer;

....

A ();

....

A ();

Desplazamiento <u>relativo</u> donde se colocará x

Lo mismo sucederá con y

RA A

EN EJECUCIÓN...

CUANDO SE INVOCA POR <u>PRIMERA</u> VEZ A LA RUTINA A:

SI EL **RA A** SE CARGA EN LA ZONA DE MEMORIA **1000** ENTONCES LA **VARIABLE x** TOMARÁ LA **DIRECCIÓN ABSOLUTA 1004**

CUANDO SE INVOCA POR <u>SEGUNDA</u> VEZ A LA RUTINA A

SI EL **RA A** SE CARGA EN LA ZONA DE MEMORIA 2050, ENTONCES LA **VARIABLE x** TOMARÁ LA **DIRECCIÓN ABSOLUTA 2054**

C3: Ejemplo del factorial

```
int n;
int fact()
   {
   int loc;  if
   (n>1) {
                 loc= n--;
                 return    loc * fact();
               }
             else return 1;
   }
main ()
   {
   get (n);
    if (n>0)    print (fact());
           else print( “input error”);
}
```

Recursión

+

Valor de retorno

C3: Funcionamiento

P [int x]

Una invocación a **P** implica → **Un molde** del RA de P para **<u>cada activación</u>** →

0	**Pto Retorno**
1	x

¿Valor de RETORNO?

En el caso C2:

D(P,0) → **pto de retorno**

D(P,1) → **x**

¿Qué información necesitaríamos adicionar en el RA en este caso?

C3: Funcionamiento

- Ahora hay que tener en cuenta que:
  - Las unidades pueden devolver **valores** (funciones) y esos valores **NO** deberían perderse cuando se desactive la unidad... Pero también que..
  - Cuando **la instancia actual** de la unidad **termine** de ejecutarse, **su registro de activación no se necesitará más**, por lo tanto se puede **liberar el espacio ocupado** por su registro de activación y dejar el espacio disponible para nuevos registros de activación Entonces....

C3: Datos necesarios

Para manejar la **alocación dinámica en la pila de una unidad** necesitamos nuevos elementos:

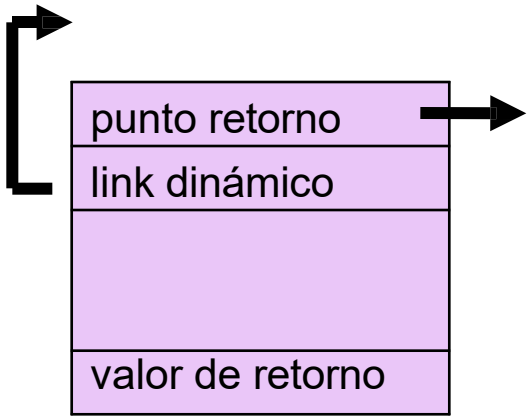
- **Valor de retorno:** Al terminar una rutina se desaloca su RA, por lo tanto la rutina lllamante debe guardar en su RA el **valor de retorno** de la rutina llamada.
- **Link dinámico:** contiene un puntero a la dirección base del registro de activación de la rutina llamadora

Pero además …. se necesita:

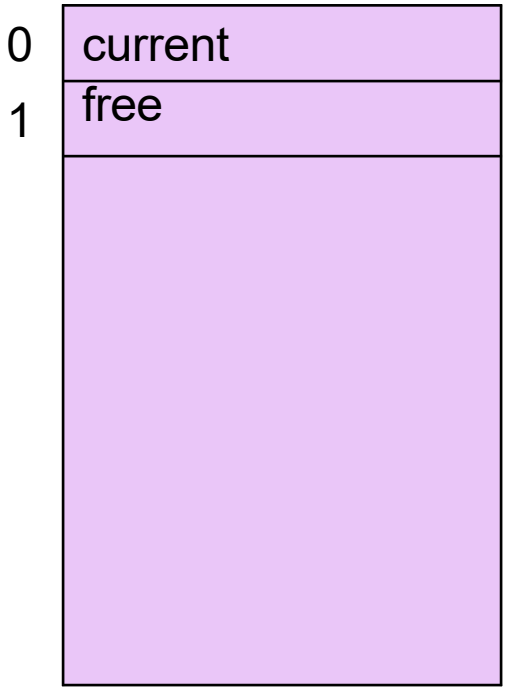
- **Current:** Dirección base del registro de activación de la unidad que se esté ejecutando actualmente
- **Free:** Próxima dirección libre en la pila

Cadena dinámica: cadena de links dinámicos originada en la secuencia de registros de activación activos.
Representa la secuencia dinámica de unidades activadas

C3: Molde de RA



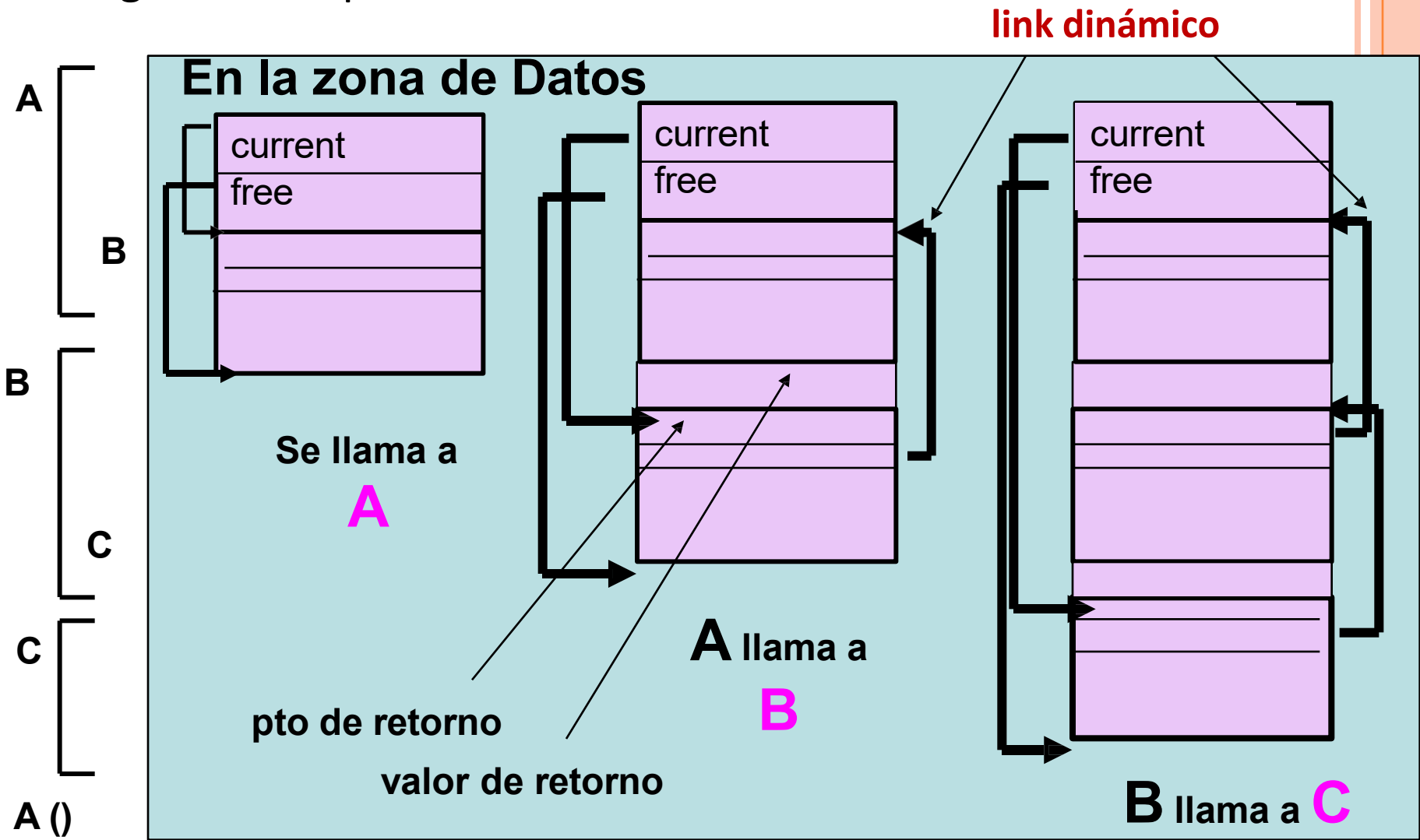
Para cada RA o instancia de ejecución de la rutina



D

C3: Ejemplo

Supongamos la siguiente estructura de programa y el siguiente esquema de llamado.



C3: ¿Qué sucede cuando se llama o retorna de una rutina?

call

free

free + 1

set 1,D[1]+1

set D[1],ip+4

set D[1]+1,D[0]

set 0,D[1]

set 1,D[1]+ size

ip+4

jump comienzo

Aloca espacio para el v. de retorno

Salva el punto de retorno al código

Salva el link dinámico

Actualiza el current

Actualiza el free

Transfiere el control a la unidad llamada

0	current
1	free
……	
Valor de retorno	
Pto retorno	
Link dinámico	
……	

return

set 1,D[0]

set 0,D[D[0]+1]

jump D[D[1]]

Reestablece el free

Reestablece el current

Bifurca al punto de retorno

link dinamico

C3: SEMÁNTICA DEL CALL - RETURN

Cuando se llama…

```
set 1,D[1]+1
set D[1],ip+4
set D[1]+1,D[0]
set 0,D[1]
set 1,D[1]+ size
jump comienzo
```

Cuando se retorna…

```
set 1,D[0]
set 0,D[D[0]+1]
jump D[D[1]]
```

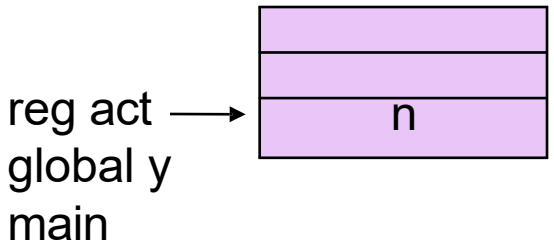
```
int f1()
 {
   int a;
      ...
 }
void main ()
 {
   int b;
   ...
   f1();
   ..
 }
```

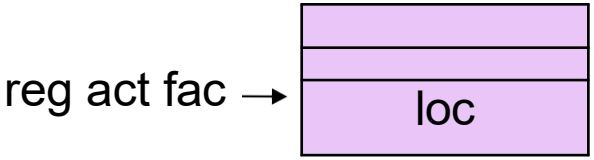
C3: Ejemplo del factorial

Basándonos en el primer ejemplo: llamada recursiva a la función factorial tenemos:

```
int n;
int fact()
{
    int loc;
    if (n>1) {
        loc = n--;
        return loc*fact();
    }
    else
        return 1;
}
```

```
main()
{
    get(n);
    if (n>=0)
        print(fact());
    else
        print("error entrada");
    }
```



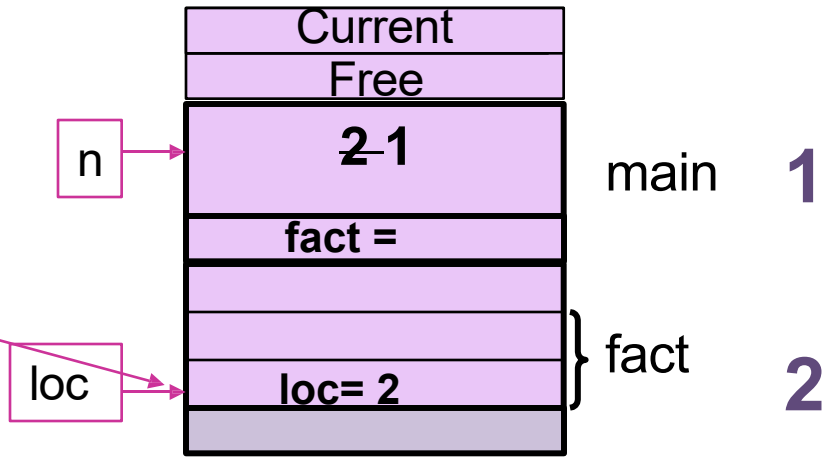


C3: Ejemplo del efecto en memoria de la ejecución de factorial

main fac fac

```
int n;
int fact()
{
        int loc;
        if (n>1) {          2
                loc = n--;
                return loc*fact();
                }
        else            return 1;
}
main()
{
    get(n);             1
    if (n>=0)
                print(fact());
    else                    print("error
    entrada");
}
```

Invocación a fac con n=2

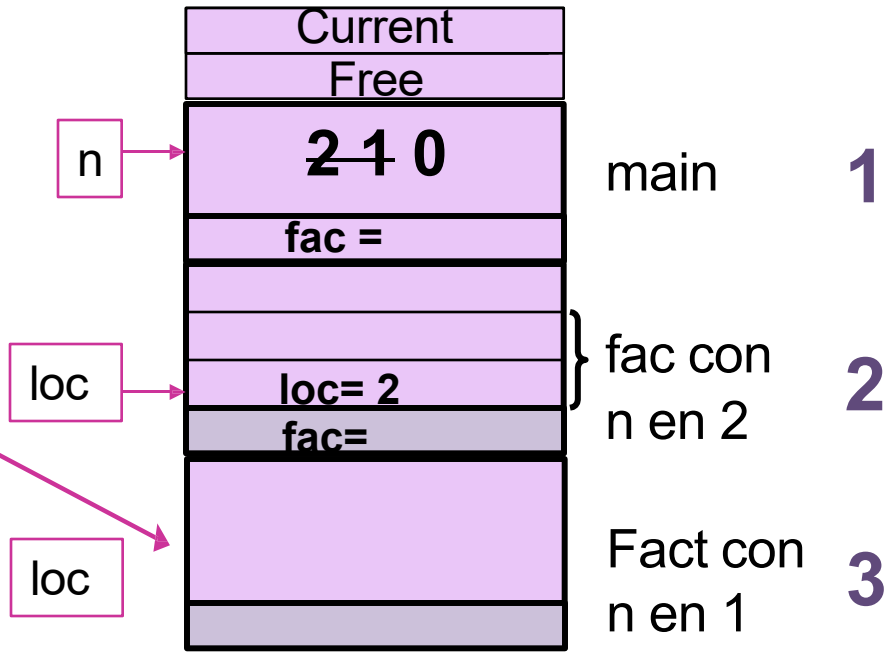


C3: Ejemplo del efecto en memoria de la ejecución de factorial

main fac fac

```
int n;
int fact()
{
        int loc;
        if (n>1) {        2    3
                loc = n--;
                return loc*fact();
                 }
        else            return 1;
}
main()
{
     get(n);              1
     if (n>=0)
                print(fact());
     else                    print("error
     entrada");
}
```

Invocación a fac con n=2

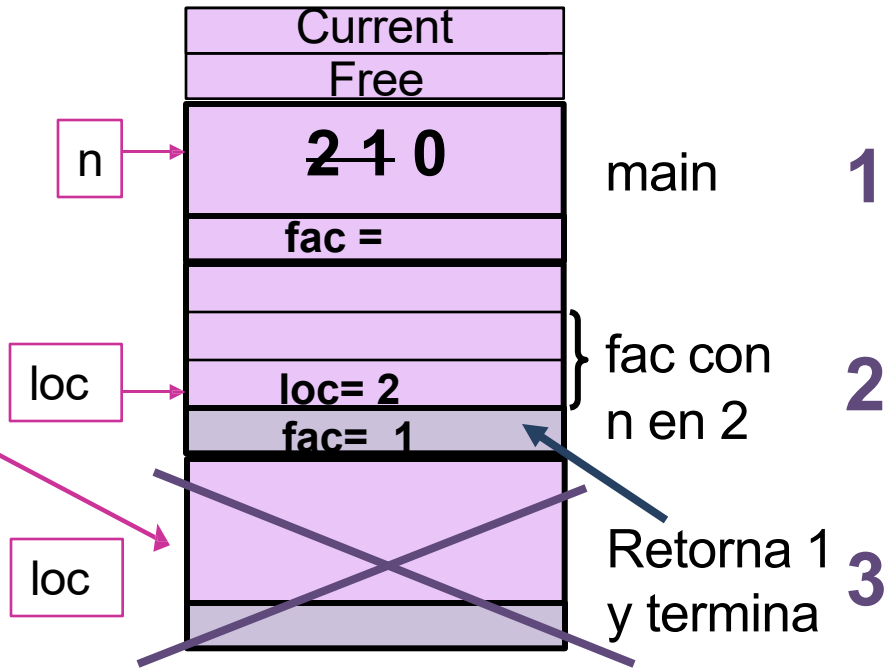


C3: Ejemplo del efecto en memoria de la ejecución de factorial

main fac fac

```
int n;
int fact()
{
        int loc;
        if (n>1) {        2    3
                loc = n--;
                return loc*fact();
                 }
        else            return 1;
}
main()
{
     get(n);              1
     if (n>=0)
                print(fact());
     else                    print("error
     entrada");
}
```

Invocación a fac con n=2

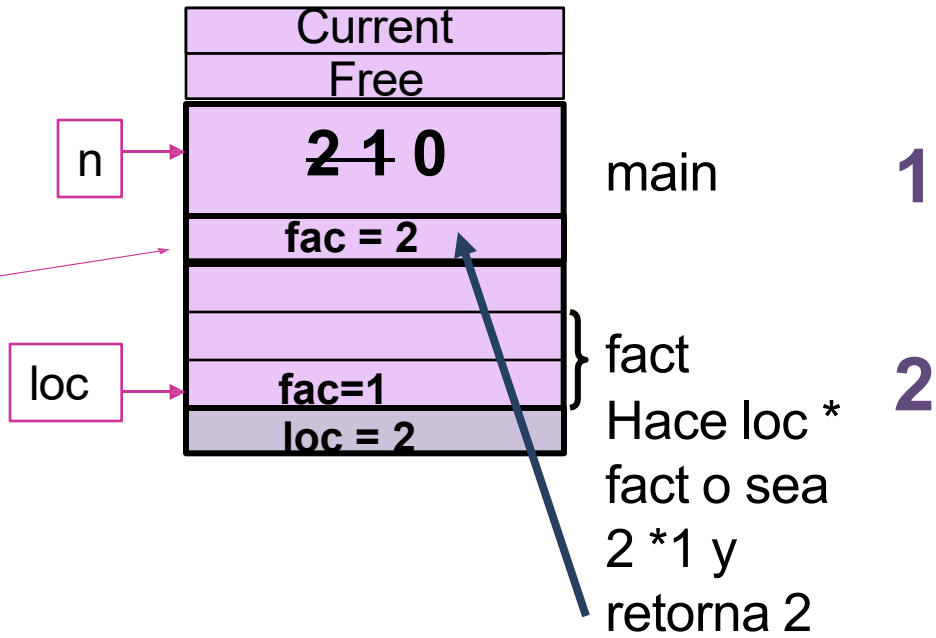


C3: Ejemplo del efecto en memoria de la ejecución de factorial

main fac fac

```
int n;
int fact()
{
        int loc;
        if (n>1) {          2
                loc = n--;
                return loc*fact();
                 }
        else          return 1;
}
main()
{
     get(n);                1
     if (n>=0)
                print(fact());
     else                    print("error
     entrada");
}
```

Invocación a fac con n=2



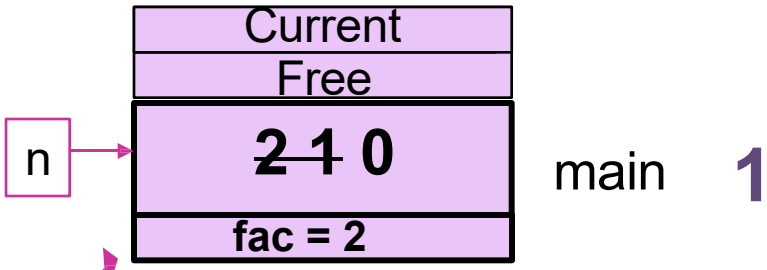
C3: Ejemplo del efecto en memoria de la ejecución de factorial

main fac fac

```
int n;
int fact()
{
        int loc;
        if (n>1) {
                loc = n--;
                return loc*fact();
                }
        else        return 1;
}
main()
{
    get(n);
    if (n>=0)
                print(fact());
    else                print("error
    entrada");
}
```

1

Invocación a fac con n=2



imprime 2

C4: ESTRUCTURA DE BLOQUE

C4´

permite que dentro de las sentencias compuestas aparezcan declaraciones locales

C4´´

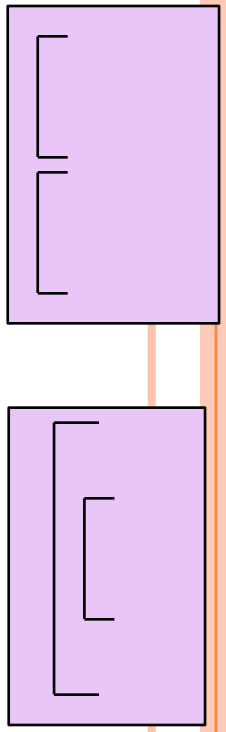
permite la definición de una rutina dentro de otras rutinas. (anidamiento de rutinas)

Estas características conforman el concepto de **estructura de bloque:**

- controla el alcance de las variables,
- define el tiempo de vida de las variables
- Divide el programa en unidades mas pequeñas.

Los bloques pueden ser:

- disjuntos (no tiene porción común)
- anidados (un bloque esta completamente contenido en otro)



C4': Anidamiento vía sentencias compuestas

- Un bloque tiene forma de una sentencia compuesta:

  {<lista de declaraciones>;<lista de sentencias>}

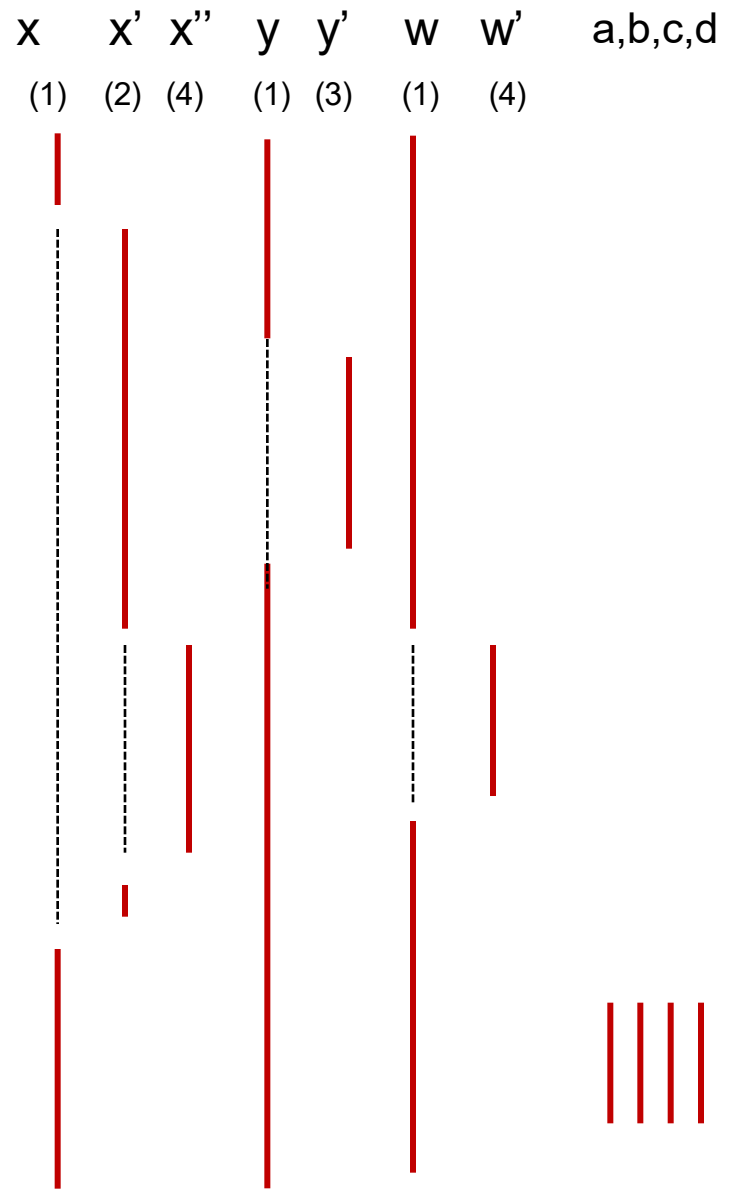
- **Las variables tienen alcance local:** son visibles dentro de la sentencia compuesta, incluyendo cualquier sentencia compuesta anidada en ella
- Si en el anidamiento, hay una nueva declaración de un nombre, **la declaración interna enmascara la externa del mismo nombre**.

C4': Sentencias compuestas

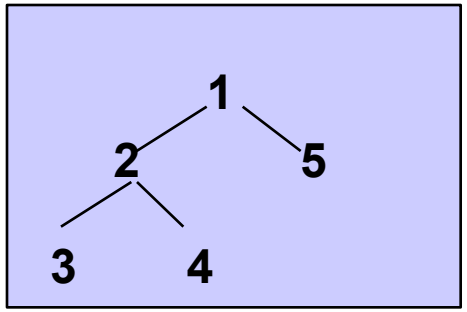
```
int f();
{
int x,y,w:
while (........)
{
 int x,z:
 while (....)
 {
  int y;
 }
 if(.....)
 {
  int x, w;
 }
}
if (...)
{
 int a,b,c,d
}
}
```

1 2 3 4 5

ALCANCE



C4': Sentencias compuestas



Alocación: implementación

estático

incluir las necesidades dentro del registro de activación de la unidad a la que pertenece

simple y eficiente en tiempo

dinámico

alocar el espacio dinámicamente cuando se ejecutan las sentencias

eficiente en espacio

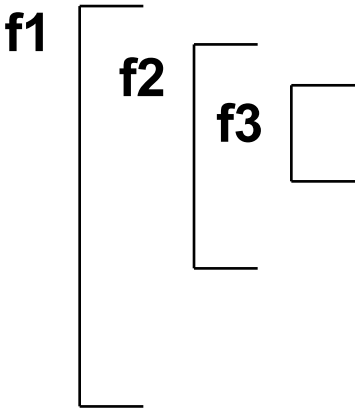
punto de retorno	
link dinámico	
x de 1	
y de 1	
w de 1	
x' de 2	a de 5
z de 2	b de 5
y' de 3 x'' de 4	c de 5
w' de 4	d de 5

1

```
int f();
{ int x,y,w:
  while (……..)
  {     int x,z:
        while (….)
        { int y;
        }
        if(…..)
        {int x, w;
        }
  }
  if (…)
  {int a,b,c,d
  }
}
```

1 2 3 4 5

C4": Rutinas anidadas




```
//file
int x,y,z;
f1()
{
    int t,u;
    f2()
    {
            int x,w;
            f3()
            {
                    int y,w,t;
            }
             x = y + t + w + z;
    }
}
```

```
main ();
{
    int z,t;

}
//end file
```

Secuencia de llamadas:

main f1 f2 f3 f2

Las variables no locales de f2 ¿a qué unidad pertenecen?

x = y + t + w + z

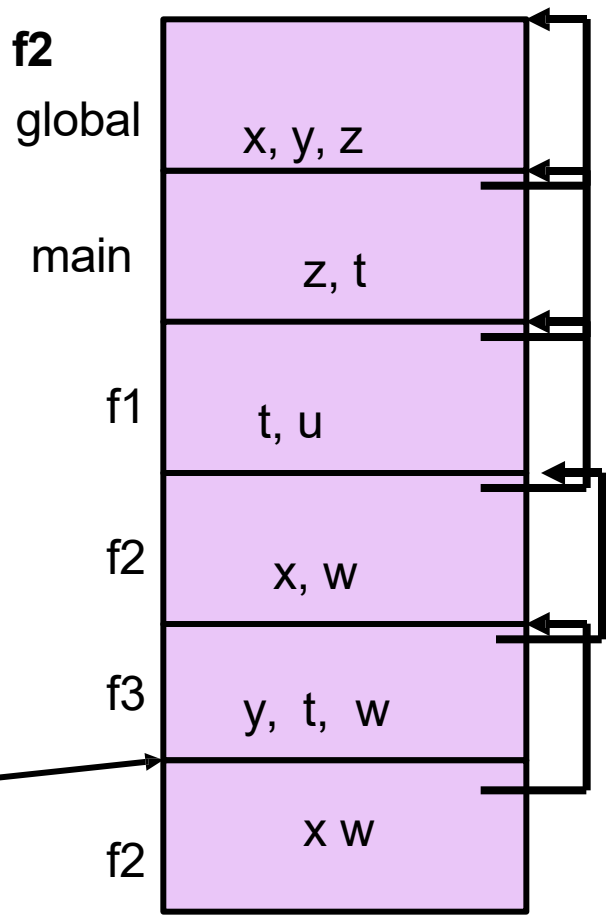
C4": Rutinas anidadas acceso al ambiente no-local por cadena DINÁMICA

```
//file
int x,y,z;
f1()
{
   int t,u;
   f2()
   {
      int x,w;
      f3()
      {
         int y,w,t;
      }
      x = y + t + w + z;
   }
}
main ();
{
int z,t;

}
//end file
```

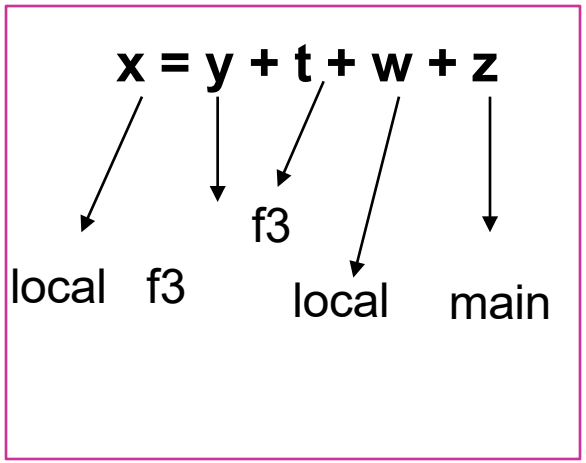
Secuencia de llamadas:

main f1 f2 f3 f2



Links dinámicos

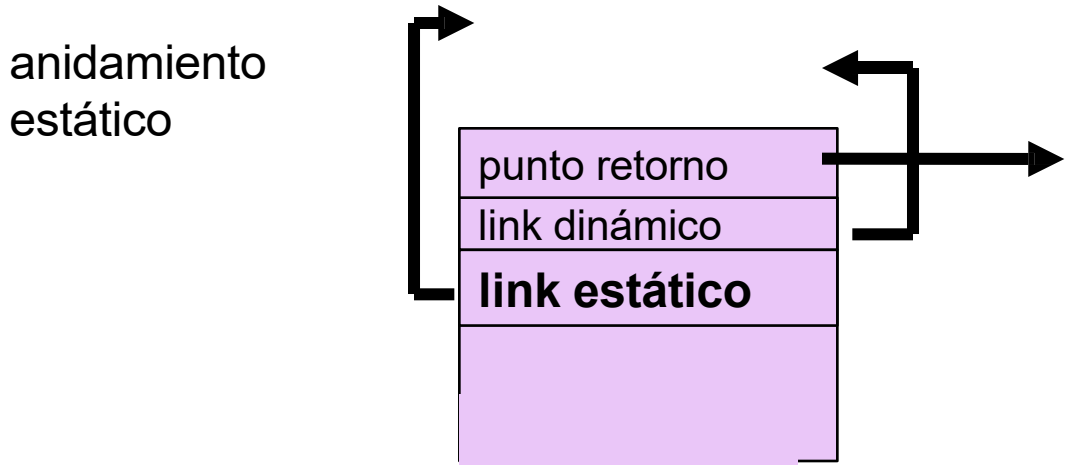
cadena dinámica



Link dinámico: localiza referencias no locales en lenguajes que siguen la cadena dinámica

C4": Acceso al ambiente no local por cadena estática

- **Link estático:** apunta al registro de activación de la unidad que estáticamente la contiene



- La secuencia de **links estáticos** se denomina **cadena estática**

C4'': ¿Qué sucede cuando se llama o retorna de una rutina?

call

free

free + 1

set 1,D[1]+1	aloca espacio para el v. de retorno
set D[1],ip + 5	salva el punto de retorno
set D[1]+1,D[0]	salva el link dinámico
set 0,D[1]	
set 1,D[1]+ size	
ip+ 5 jump comienzo	
	llamada

Se agrega una sentencia:

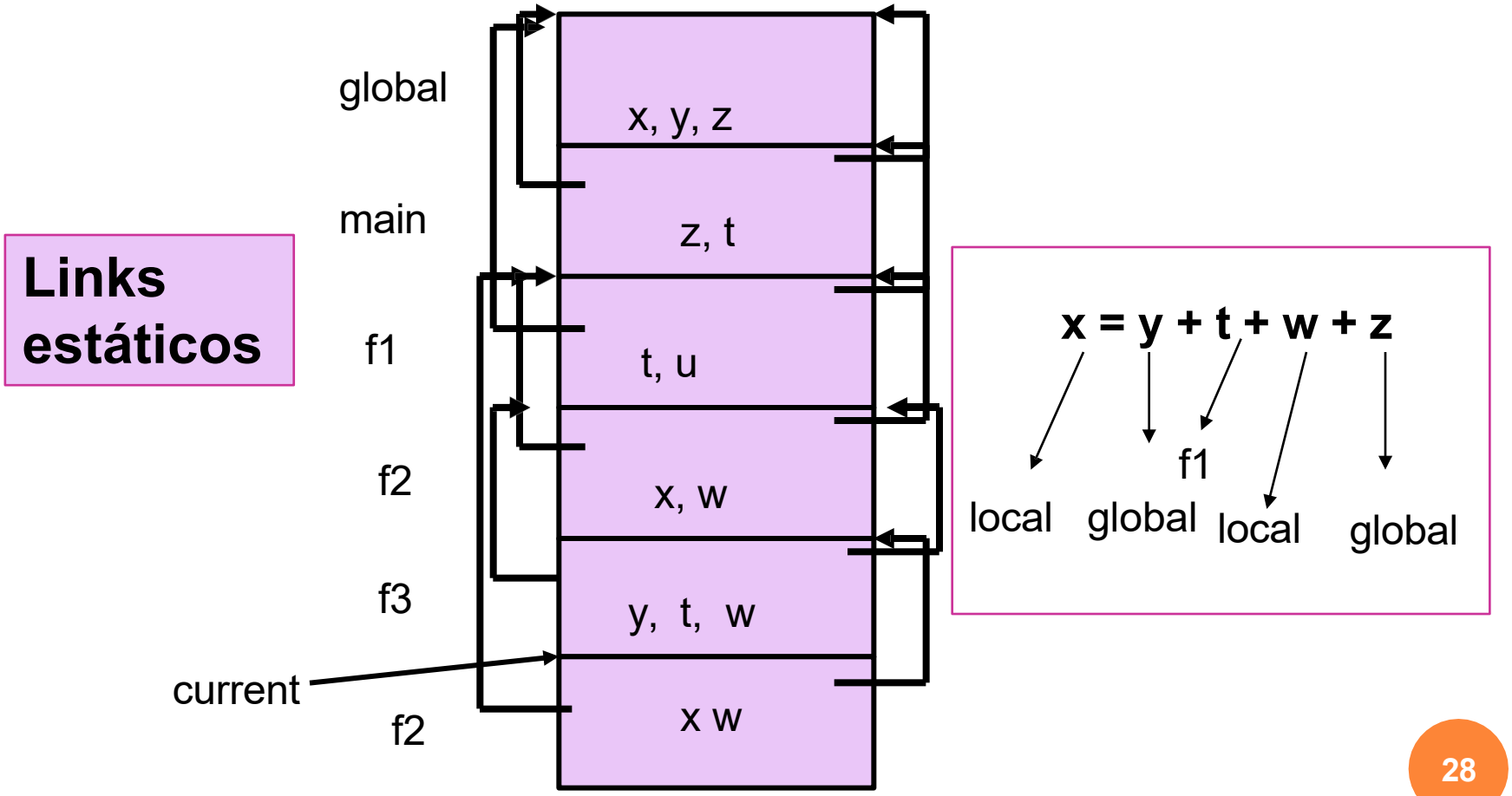
Colocar en D[1] + 2 el link estático

set 1,D[0]	reestablece el free
set 0,D[D[0]+1]	reestablece el current
jump D[D[1]]	bifurca al punto de retorno

link dinamico

return

C4": Cadena estática



Link estático: sirve para localizar referencias no locales en lenguajes que siguen la **cadena estática**

Cadena estática ≠ Cadena dinámica

A
- boolean b= true
- B
  - print b
- C
  - boolean b= false
  - B

- A C B
  - estático
    - b es la de A → true
  - dinámico
    - b es la de C → false

C5: Datos más dinámicos

C5´

Registro de activación cuyo tamaño se conoce cuando se activa la unidad.

Datos semidinámicos

C5´´

Los datos pueden alocarse durante la ejecución.

Datos dinámicos

C5': Datos semidinámicos

Variables cuyo tamaño se conoce en compilación

- **Arreglos dinámicos**

type VECTOR is array (INTEGER range <>);

define un arreglo con **índice irrestricto**

A: VECTOR (0..N);

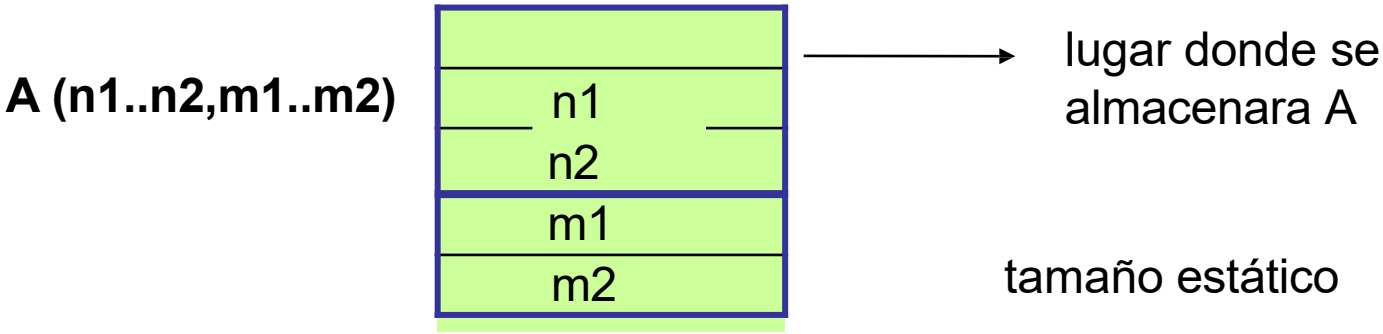
B: VECTOR(1..M);

N y M deben ligarse a algún valor entero para que A y B puedan alocarse en ejecución (referencia al ambiente no local o parámetros)

C5': Datos semidinámicos

Implementación de arreglos dinámicos

- En **compilación** se reserva lugar en el registro de activación para los **descriptores** de los arreglos dinámicos.

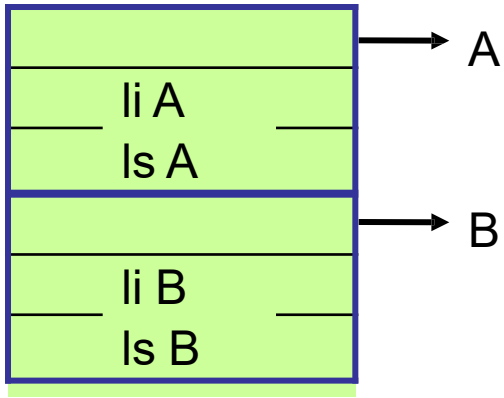


Todos los accesos al arreglo dinámico son traducidos como referencias indirectas a través del puntero en el descriptor, cuyo desplazamiento se determina estáticamente.

C5': Datos semidinámicos

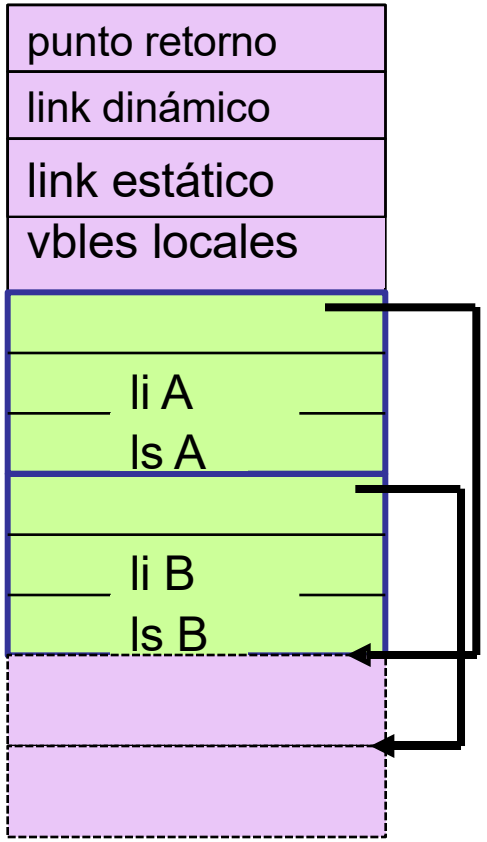
Implementación de arreglos dinámicos

A: VECTOR (0..N);
B: VECTOR(1..M);



COMPILACIÓN

ACTIVACIÓN DE LA UNIDAD SE EVALUAN LOS LIMITES



código generado por el compilador para implementar este mecanismo

EJECUCIÓN

C5': Datos semidinámicos

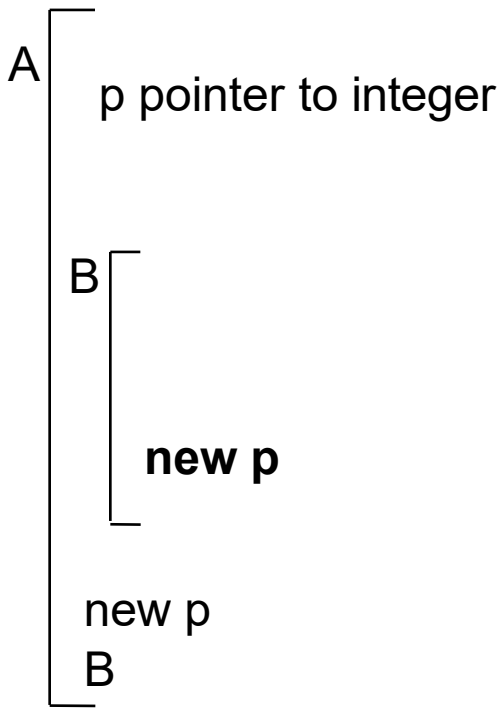
EJECUCIÓN: el registro de activación se aloca en varios pasos:

- Se aloca el almacenamiento para los datos de tamaño conocido estáticamente y para los descriptores de los arreglos dinámicos.
- Con la declaración se calculan las dimensiones en los descriptores y **se extiende el registro de activación para incluir el espacio para la variable dinámica**.
- Se fija el puntero del descriptor con la dirección del área alocada

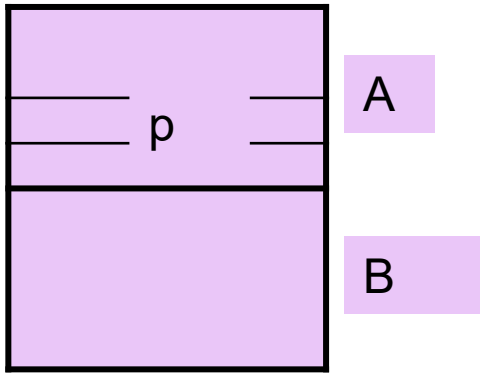
C5": Datos dinámicos

- Se alocan explícitamente durante la ejecución mediante instrucciones de alocación

¿dónde alocar el objeto apuntado?



A B

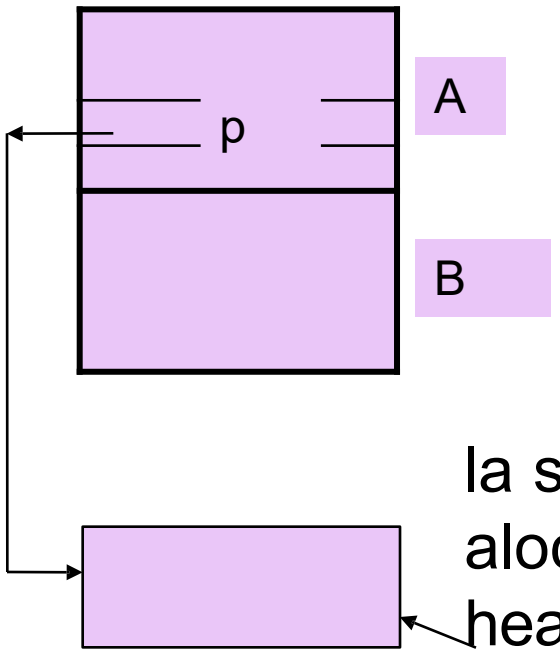


el reg de Act de A no está en el tope

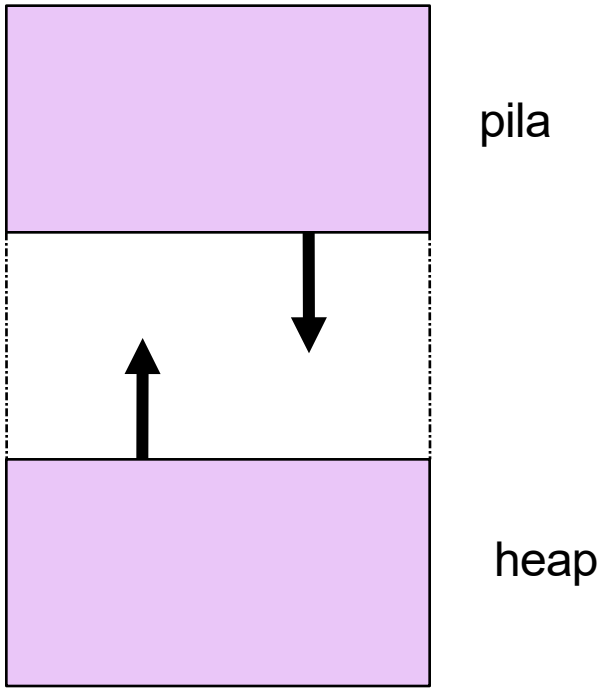
el reg de Act de B no permite que luego se use en A

C5”: Datos dinámicos

EL TIEMPO DE VIDA NO DEPENDE DE LA SENTENCIA DE ALOCACIÓN, VIVIRÁ MIENTRAS ESTÉ APUNTADA



la solución es alocarla en la heap



Esquemas de ejecución
Casos de C3 a C6

- Estático
- Basado en pila
- Dinámico

C6: LENGUAJES DINÁMICOS

- Se trata de aquellos lenguajes que adoptan más reglas dinámicas que estáticas.
- Usan **tipado** dinámico y reglas de **alcance** dinámicas.
- Se podrían tener reglas de tipado dinámicas y de alcance estático, pero en la práctica las propiedades dinámicas se adoptan juntas.
- Una propiedad dinámica significa que las ligaduras correspondientes se llevan a cabo en ejecución y no en compilación.

Esquemas de ejecución de las unidades y variables

- Casos C1 y C2
- Variables estáticas

estático

- Casos C3, C4 y C5
- Variables semiestáticas o automáticas
- Variables semidinámicas de C5’

pila

- Caso C6
- Variables dinámicas de C5’’
- Tipos y alcance dinámico C6

heap

Esquemas de ejecución de las unidades

- Variables estáticas C1-C2

estático

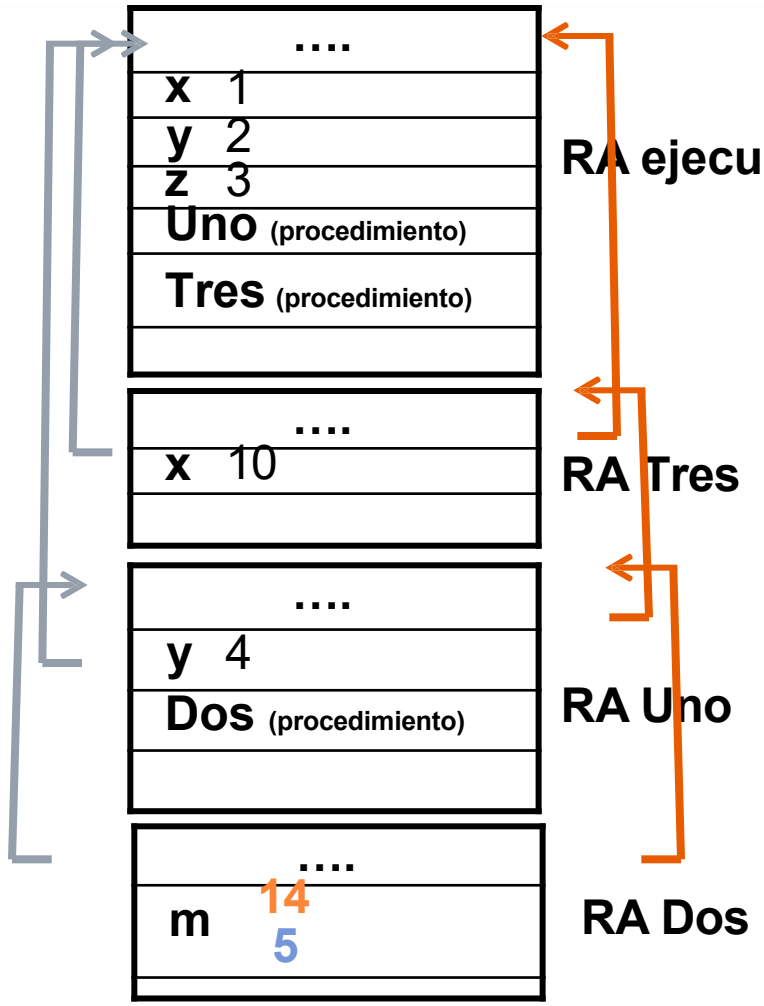
- Variables semiestáticas o automáticas C3-C4
- Variables semidinámicas C5'

pila

- Variables dinámicas C5''
- Tipos y alcance dinámico C6

heap

```
1  Program ejecucionRutina(output);
2      var
3          x , y , z: integer;
4      procedure Uno();
5          var
6              y :integer;
7          procedure Dos();
8              var
9                  m:integer;
10             begin
11                 m:= x+y;
12                 writeln(m);
13             end;
14         begin
15             y :=4;
16             Dos();
17         end;
18     procedure Tres();
19         var
20             x:integer;
21         begin
22             x:=10;
23             Uno();
24         end;
25     begin
26         x:=1;     y:=2;     z:=3;
27         writeln('estamos probando ejecución de rutinas!');
28         Tres();
29         writeln(x,y,z)
30     end.
```



Resultados por:

Cadena Dinámica

Cadena Estática

Ejemplo de cadena dinámica y estática